FIAP

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Fabio Cezare Almeida, RM 572642
Guilherme Zanfolim Nunes Farias, RM 570983
Jaime Ringel, RM 562044
Lucas Monteiro Dias da Costa, RM 571388
Matheus Magalhães Romão de Moraes, RM 573371

Domain Driven Design Using Java

Sprint 4 - Entrega final da API Restful

Equipe: EcoMovement

EcoMovement - API Restful de missões sustentáveis

SÃO PAULO

2026

1. Objetivo e escopo do projeto..... 3
Escopo desta Sprint.....3
2. Principais funcionalidades..... 4
3. Arquitetura e tecnologias.....5
Camadas do projeto.....5
4. Tabela de endpoints (API Restful).....6
Códigos de resposta..... 7
5. Protótipo — telas implementadas.....8
6. Modelo Entidade-Relacionamento (MER).....14
Diagrama do banco Oracle com todas as tabelas..... 14
7. Diagrama de classes..... 15
Camadas: Resource → BO → DAO → ConnectionFactory..... 15
8. Regras de negócio e boas práticas.....16
Regras de negócio (camada BO)..... 16
Boas práticas aplicadas.....16
Limitações conhecidas.....16
9. Procedimentos para rodar a aplicação.....17
Ferramentas e versões.....17
Passo a passo..... 17
Usuários de demonstração.....17
10. Deploy.....19
Variáveis de ambiente configuradas no servidor..... 19

1. Objetivo e escopo do projeto

O EcoMovement é uma plataforma que transforma pequenas ações sustentáveis do dia a dia em pontos (EcoPoints). O usuário escolhe uma missão no site, envia uma foto ou vídeo como comprovação e a equipe EcoGuard aprova ou rejeita o envio. Quando a evidência é aprovada, o usuário recebe os pontos e sobe no ranking.

Esta entrega corresponde à API Restful em Java com Spring Boot que atende o front-end em React. Ela guarda os dados em banco Oracle, nas tabelas do MER do grupo (prefixo T_ECMV_), usando JDBC puro com os padrões DAO e Connection Factory (sem JPA).

Escopo desta Sprint

Dentro do escopo: módulo de Missões, envio e validação de evidências, EcoPoints, ranking, login do Painel EcoGuard, cadastro de usuário e formulário de contato.

Fora do escopo desta entrega: demais tabelas do MER (árvore digital, floresta, badges, premiação, QR Code, análise por IA etc.), que existem no banco mas ainda não têm endpoints.

2. Principais funcionalidades

Funcionalidade	Descrição	Status
CRUD de missões	Cadastro, consulta, atualização e exclusão de missões, com imagem ilustrativa (BLOB).	Implementada
Lista de missões por usuário	Missões ativas com status (disponível, em análise, concluída) e progresso do usuário.	Implementada
Envio de evidência	Upload de foto ou vídeo (até 50 MB) validado conforme o tipo da missão e o tempo máximo do vídeo.	Implementada
Validação EcoGuard	Aprovação ou rejeição do envio; ao aprovar, registra a validação e credita os EcoPoints.	Implementada
EcoPoints e ranking	Saldo do usuário e ranking calculados a partir dos movimentos de pontos; empates dividem a posição.	Implementada
Histórico e reinício de testes	Histórico de envios do usuário e opção de reiniciar o fluxo de teste.	Implementada
Cadastro de usuário	Cria usuários com perfil USUARIO, valida os dados (nome, e-mail e senha) e impede e-mail repetido.	Implementada
Login do Painel EcoGuard	Autenticação do administrador com senha em hash SHA-256 e verificação do perfil ADMIN.	Implementada
Fale conosco	Grava, lista, consulta e exclui as mensagens do formulário de contato.	Implementada
Documentação interativa	Swagger UI (springdoc) para consultar e testar todos os endpoints.	Implementada

3. Arquitetura e tecnologias

Tecnologia	Uso
Java 21	Linguagem
Spring Boot 3.5 (Spring Web)	Framework da API Restful
JDBC + ojdbc11	Acesso ao Oracle (DAO + Connection Factory, sem JPA)
Oracle Database (FIAP)	Banco de dados relacional
springdoc-openapi 2.8	Swagger UI em /swagger-ui.html
Maven 3.9+	Build e dependências
Docker e Render	Empacotamento e hospedagem da API em nuvem

Camadas do projeto

O caminho de uma requisição é: Resource recebe → BO valida as regras → DAO executa o SQL → Oracle.

Pacote	Responsabilidade
model	Classes de modelo: Missao, Submissao, Evidencia, ValidacaoEcoguard, MovimentoEcopoints, ProgressoMissao, Usuario, NovoUsuario, Contato, LoginAdmin, PosicaoRanking, Arquivo.
dao	Um DAO por tabela, com o SQL (PreparedStatement) e o CRUD.
bo	MissaoBO, SubmissaoBO, UsuarioBO, ContatoBO: validações e regras de negócio.
resource	Endpoints REST (@RestController).
factory	ConnectionFactory: abre a conexão com o Oracle.
config	CorsFilter: libera o front-end (CORS) pelas origens configuradas.
exception	RegraNegocioException, NaoEncontradoException, NaoAutorizadoException, ConflitoException e TratadorDeErros (respostas de erro em JSON).

4. Tabela de endpoints (API Restful)

Todas as respostas de erro vêm em JSON no formato {"status": 400, "mensagem": "..."}. Qualquer endpoint que acesse o banco pode responder 500 em caso de falha de banco de dados ou de erro inesperado no servidor; nesse caso a mensagem é genérica e o detalhe técnico fica apenas no log do servidor.

Verbo	URI	Descrição	Status esperados
GET	`/`	Status da API (online) e caminho do Swagger	200
GET	`/missoes?usuarioId={id}`	Missões ativas com status e progresso do usuário	200, 400, 404, 500
GET	`/missoes`	Catálogo completo (inclui inativas)	200, 500
GET	`/missoes/{id}?usuarioId={id}`	Detalhe de uma missão	200, 404, 500
POST	`/missoes`	Cadastra uma missão (corpo JSON)	201, 400, 500
PUT	`/missoes/{id}`	Atualiza uma missão (corpo JSON)	200, 400, 404, 500
DELETE	`/missoes/{id}`	Exclui a missão (só se não tiver envios)	204, 400, 404, 500
GET	`/missoes/{id}/imagem`	Devolve a imagem da missão	200, 404, 500
PUT	`/missoes/{id}/imagem`	Envia a imagem (corpo binário, Content-Type image/*)	204, 400, 404, 500
POST	`/submissoes`	Envia foto/vídeo (multipart: usuarioId, missaoId, tipoEvidencia, duracaoSegundos, arquivo)	201, 400, 404, 500
GET	`/submissoes?status={status}`	Envios por status: EM_ANALISE (padrão), APROVADA, REJEITADA	200, 400, 500
GET	`/submissoes/{id}`	Detalhe de um envio	200, 404, 500
GET	`/submissoes/{id}/evidencia`	Devolve a foto/vídeo enviado	200, 404, 500
PUT	`/submissoes/{id}/aprovar`	EcoGuard aprova e credita os EcoPoints	200, 400, 404, 500
PUT	`/submissoes/{id}/rejeitar`	EcoGuard rejeita (usuário pode reenviar)	200, 400, 404, 500
DELETE	`/submissoes/{id}`	Exclui o envio e os registros ligados a ele	204, 404, 500
POST	`/usuarios`	Cadastra um usuário (corpo JSON: nome, email, senha)	201, 400, 409, 500
GET	`/usuarios/{id}`	Detalhe de um usuário	200, 404, 500
GET	`/usuarios/{id}/submissoes`	Histórico de envios do usuário	200, 404, 500
DELETE	`/usuarios/{id}/submissoes`	Reinicia o teste: apaga os envios de foto/vídeo do usuário	200, 404, 500
GET	`/usuarios/{id}/pontos`	Saldo de EcoPoints: {total: 20}	200, 404, 500
GET	`/ranking`	Ranking por EcoPoints (posicao, usuarioId, nome, pontos)	200, 500

Verbo	URI	Descrição	Status esperados
POST	`/admin/login`	Login do Painel EcoGuard ({"email", "senha"})	200, 400, 401, 500
POST	`/contatos`	Salva a mensagem do formulário de contato	201, 400, 500
GET	`/contatos`	Lista as mensagens recebidas	200, 500
GET	`/contatos/{id}`	Detalhe de uma mensagem de contato	200, 404, 500
DELETE	`/contatos/{id}`	Exclui uma mensagem de contato	204, 404, 500

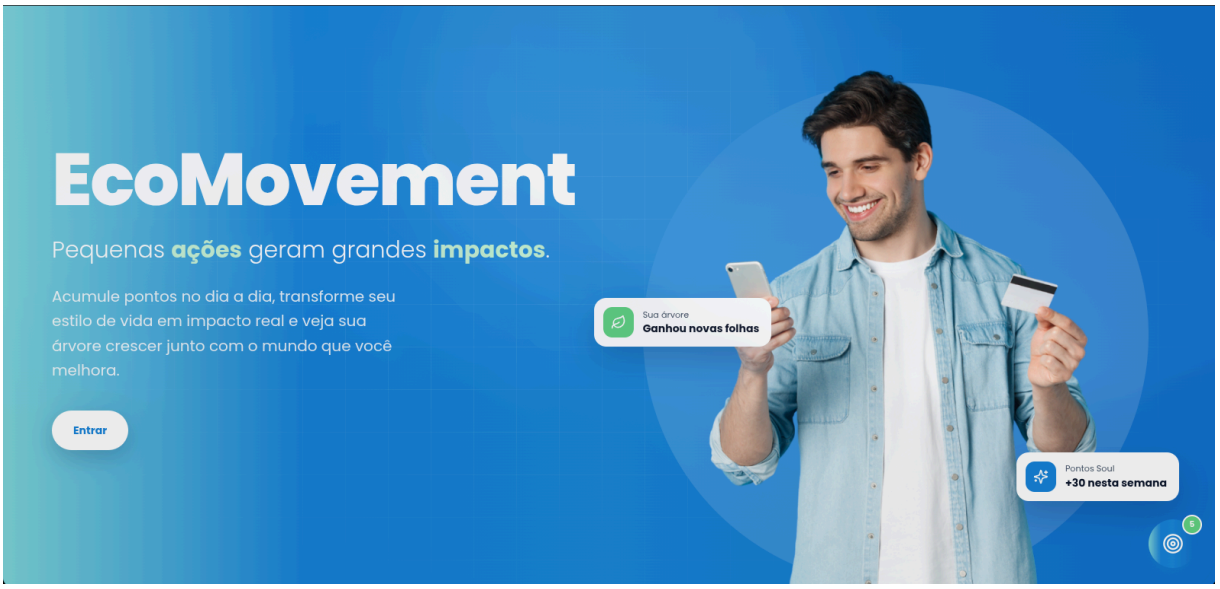
Códigos de resposta

Código	Significado na API
200 / 201 / 204	Sucesso: consulta ou alteração / recurso criado / sem conteúdo (exclusão)
400	Regra de negócio violada ou requisição inválida (RegraNegocioException, parâmetro ausente, JSON malformado, arquivo grande)
401	E-mail ou senha incorretos, ou usuário sem perfil ADMIN
404	Recurso não encontrado (NaoEncontradoException) ou rota inexistente
405	Método HTTP não permitido para o endereço acessado
409	Conflito: já existe um usuário cadastrado com o e-mail informado (ConflitoException)
415	Tipo de conteúdo (Content-Type) não suportado pelo endpoint
500	Erro de banco de dados (SQLException) ou erro inesperado no servidor

5. Protótipo — telas implementadas

Capturas reais do sistema (front-end React consumindo esta API publicada em nuvem).

Figura 1 — Tela inicial do site EcoMovement. Apresenta a proposta da plataforma e o botão “Entrar”.



Fonte: Elaborado pelos autores (2026)

Figura 2 — Tela de Missões. Lista as missões ativas com status e progresso do usuário (GET /missoes?usuarioId=1).



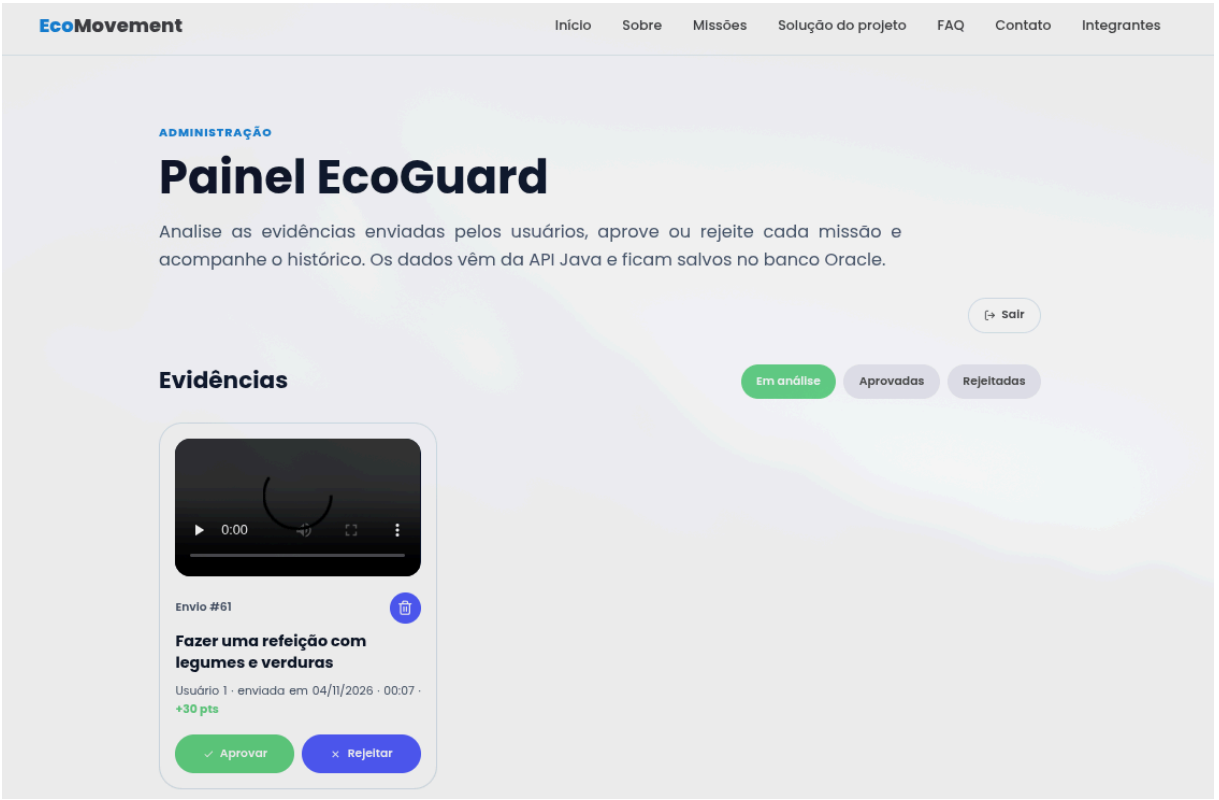
Fonte: Elaborado pelos autores (2026)

Figura 4 — Envio de evidência. Upload da foto ou do vídeo da missão (POST /submissoes).



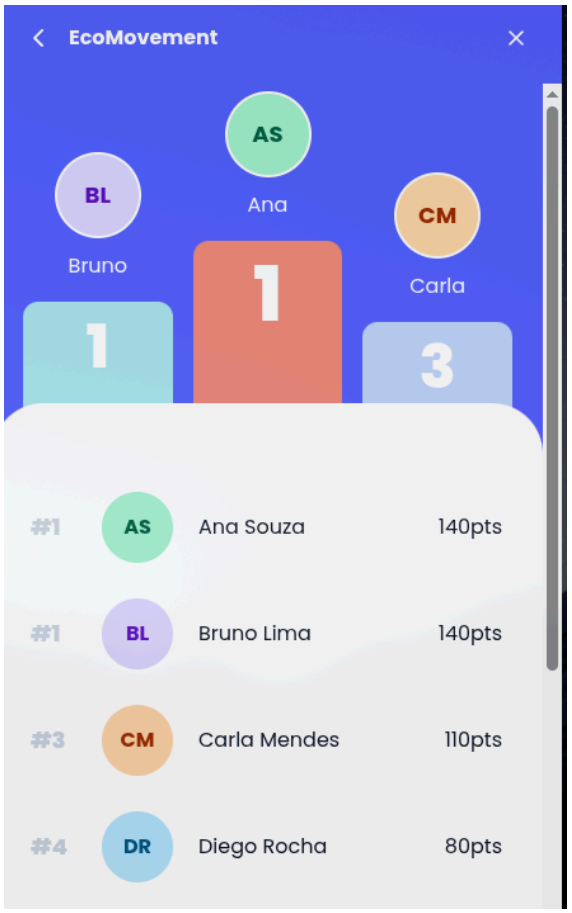
Fonte: Elaborado pelos autores (2026)

Figura 5 — Painel EcoGuard — Em análise. Revisão das evidências e botões Aprovar/Rejeitar (PUT /submissoes/{id}/aprovar).



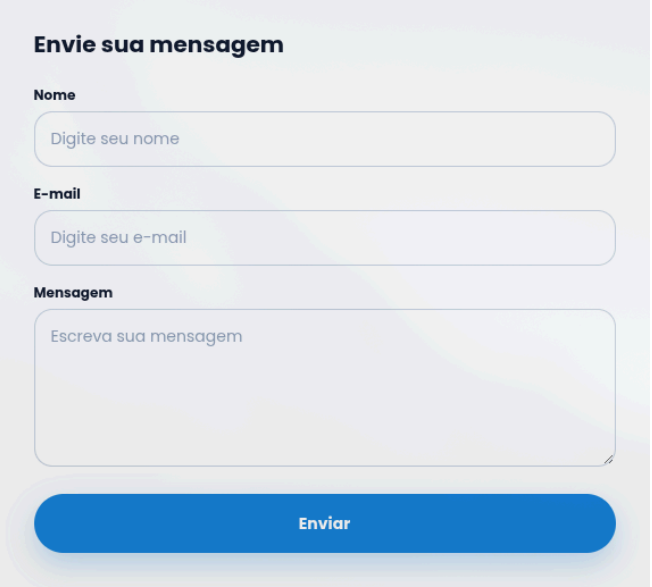
Fonte: Elaborado pelos autores (2026)

Figura 6 — Ranking e EcoPoints. Posição do usuário e saldo de pontos (GET /ranking e /usuarios/{id}/pontos).



Fonte: Elaborado pelos autores (2026)

Figura 7 — Fale conosco. Formulário de contato salvo no banco (POST /contatos).

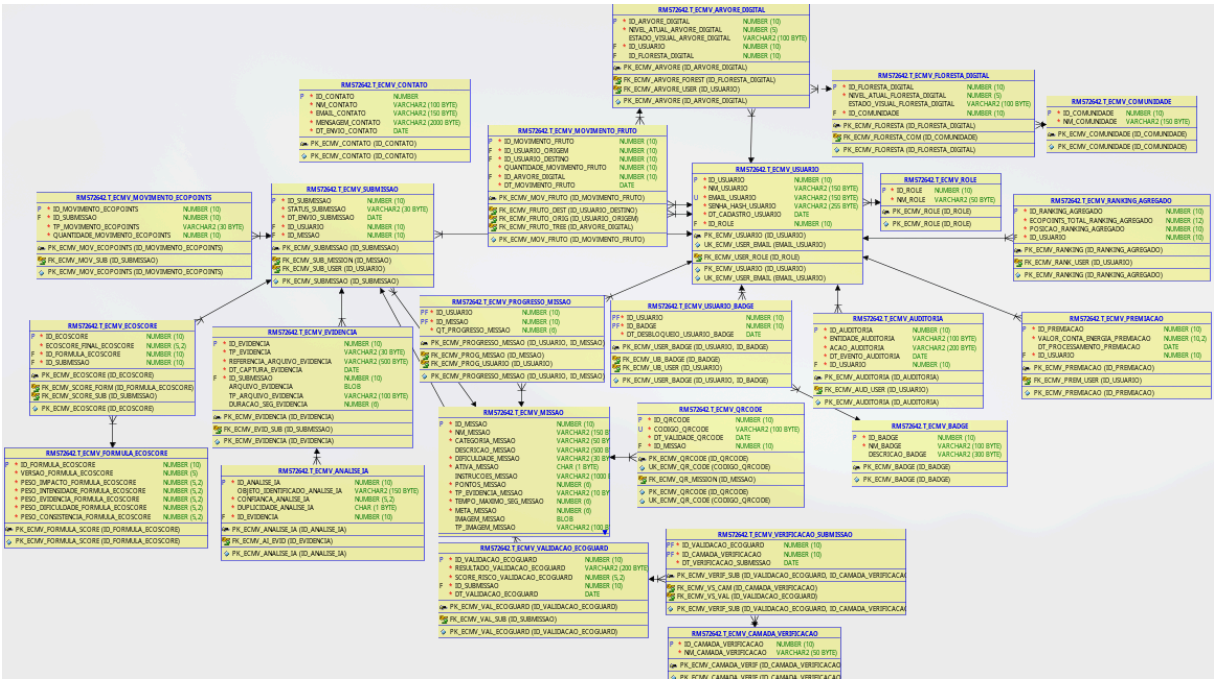


Fonte: Elaborado pelos autores (2026)

6. Modelo Entidade-Relacionamento (MER)

Diagrama do banco Oracle com todas as tabelas

Figura 11 — MER do banco de dados (prefixo T_ECMV_).

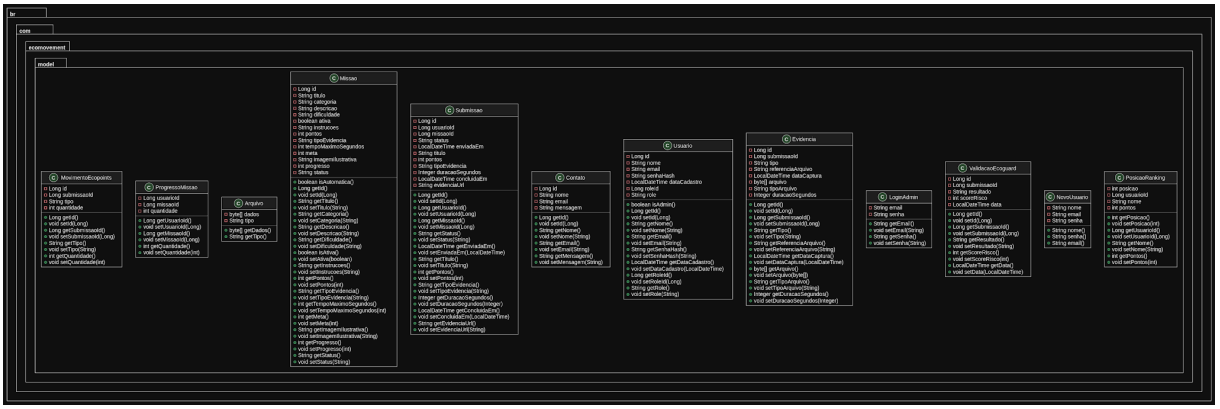


Fonte: Elaborado pelos autores (2026)

7. Diagrama de classes

Classes de modelo (pacote model). Como o acesso é por JDBC, os relacionamentos entre as classes são feitos pelos ids (chaves estrangeiras), espelhando o MER.

Figura 13 — Diagrama de classes do pacote model.

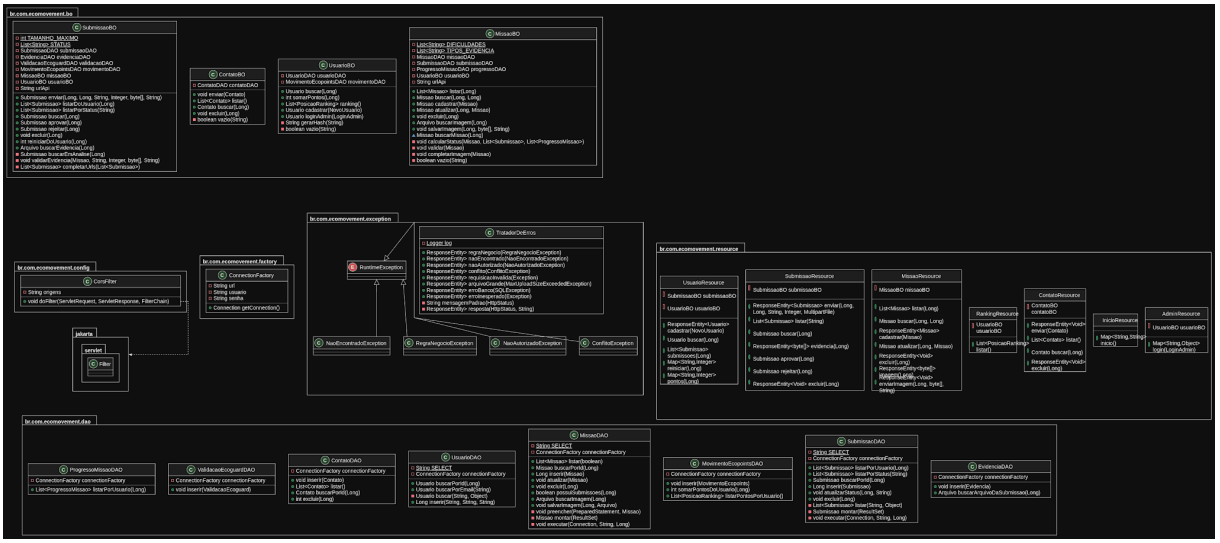


Fonte: Elaborado pelos autores (2026)

Camadas: Resource → BO → DAO → ConnectionFactory

classes das camadas Resource, BO, DAO, fábrica de conexão, exceções e CORS. Setas cheias: uso entre camadas; pontilhadas: conexão com o banco.

Figura 14 — Diagrama de classes das camadas Resource, BO, DAO, ConnectionFactory, exceções e CorsFilter.



Fonte: Elaborado pelos autores (2026)

8. Regras de negócio e boas práticas

Regras de negócio (camada BO)

- Missão de FOTO só aceita imagem; missão de VIDEO só aceita vídeo (até 50 MB). O vídeo não pode passar do tempo máximo da missão.
- Não é possível reenviar uma missão que está em análise ou já foi aprovada.
- Missões AUTOMATICAS não aceitam envio: o progresso vem da plataforma SoulUp.
- Ao aprovar, a API grava a validação (T_ECMV_VALIDACAO_ECOGUARD) e o crédito de pontos (T_ECMV_MOVIMENTO_ECOPOINTS). Os pontos do usuário são a soma desses movimentos.
- O ranking soma os movimentos de cada usuário (exceto ADMIN); empates dividem a mesma posição.
- Missão com envios não pode ser excluída (preserva o histórico): deve ser desativada.
- Validação dos dados da missão: título (até 150 caracteres), categoria, dificuldade (FACIL, MEDIA ou DIFICIL), tipo de evidência, pontos entre 1 e 1000, imagem de até 5 MB.
- Cadastro de usuário: nome de 2 a 150 caracteres, e-mail válido e único (repetido responde 409) e senha de 6 a 72 caracteres, salva em hash SHA-256 com perfil USUARIO.
- O login do admin confere o hash SHA-256 salvo no banco e exige o perfil ADMIN. O contato valida campos obrigatórios, e-mail e limite de 2000 caracteres.

Boas práticas aplicadas

- Padrão DAO + Connection Factory: todo o SQL fica nos DAOs, que usam PreparedStatement (evita SQL injection).
- Camadas MVC/REST: Resource (HTTP), BO (regras) e DAO (persistência) separados.
- Tratamento de exceções centralizado no TratadorDeErros (@RestControllerAdvice), com status HTTP corretos, mensagem em JSON e um handler genérico para erros inesperados.
- Registro de erros com Logger (SLF4J): o detalhe técnico de falhas fica no log do servidor e não é enviado ao cliente.
- CorsFilter: libera apenas as origens configuradas na variável CORS_ORIGINS.
- Configuração por variáveis de ambiente (DB_URL, DB_USER, DB_PASSWORD...), sem senhas no código; o .env está no .gitignore.
- Nomenclatura em português para o domínio e pacotes organizados por responsabilidade.

Limitações conhecidas

- Os endpoints de administração (aprovar, rejeitar, excluir, cadastrar missão) não exigem token: o login valida o perfil ADMIN, mas a API não emite nem verifica token nas demais rotas.
- As senhas usam SHA-256 sem sal, solução simples adotada para esta etapa; em produção o recomendado é BCrypt.
- A API roda no plano gratuito do Render, que hiberna após alguns minutos sem uso; a primeira requisição pode levar cerca de 1 minuto.

9. Procedimentos para rodar a aplicação

Ferramentas e versões

Ferramenta	Versão / observação
JDK	Java 21 ou mais novo
Maven	3.9+ (ou o Maven embutido do IntelliJ)
IDE	IntelliJ IDEA (usada no projeto) ou VS Code
Banco	Oracle SQL Developer, acessando o Oracle da FIAP
Front-end (opcional)	Node.js + npm, para o site React
Testes de API (opcional)	Swagger UI (já incluído) ou Insomnia/Postman

Passo a passo

1. Importar o projeto. Descompacte o .zip (ou clone o repositório) e abra a pasta ecomovement-api no IntelliJ: ele importa o pom.xml sozinho.

2. Criar o banco. No SQL Developer, execute com F5 (Executar Script), nesta ordem: 01_criacao_tabelas.sql → 02_alteracoes_mer.sql → 03_carga_dados.sql. O 04_consultas.sql só faz consultas.

3. Configurar a conexão. Copie .env.example para .env e preencha com seu usuário e senha do Oracle: DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL; DB_USER=rm123456; DB_PASSWORD=sua_senha. O .env não vai para o GitHub.

4. Iniciar a API. No terminal: mvn spring-boot:run, ou no IntelliJ rode a classe EcomovementApplication. Quando aparecer Started EcomovementApplication, abra http://localhost:8080, http://localhost:8080/swagger-ui.html e http://localhost:8080/missoes?usuarioId=1.

5. Abrir o site. O site publicado (https://projetoecomovement.vercel.app/) já consome a API em nuvem. Para rodar localmente, baixe o front-end em https://github.com/zanfolim07/Projeto-EcoMovement e, em outro terminal: npm install, copie .env.example para .env.local, defina VITE_API_URL=http://localhost:8080 (ou a URL da API publicada) e VITE_USUARIO_ID=1, e rode npm run dev (http://localhost:5173).

6. Testar o fluxo completo. Em Missões, envie a foto ou o vídeo de uma missão → abra /admin e entre com o admin → em “Em análise”, clique em Aprovar → volte em Missões: a missão aparece concluída e os pontos aumentam.

7. Testar os endpoints. Pelo Swagger (/swagger-ui.html, botão “Try it out”) ou por um cliente HTTP como o Insomnia, conforme a tabela da seção 4.

Usuários de demonstração

Contas de teste criadas pelo script 03_carga_dados.sql, usadas apenas para demonstração.

Usuário	E-mail	Senha	Perfil
Usuário Demo (id 1)	demo@ecomovement.com	senha123	USUARIO (usado pelo site)
Admin EcoGuard	ecomovement@admin.com	1234	ADMIN (login do /admin)

10. Deploy

Item	Valor
Plataforma	Render (Web Service, plano gratuito, deploy com Docker)
URL da API	https://ecomovement-api.onrender.com
Swagger em produção	https://ecomovement-api.onrender.com/swagger-ui.html
Repositório da API (GitHub)	https://github.com/FabioCAlmeida/EcoMovement-API
Repositório do front-end (GitHub)	https://github.com/zanfolim07/Projeto-EcoMovement
Front-end publicado	https://projetoecomovement.vercel.app/

Variáveis de ambiente configuradas no servidor

Variável	Descrição
DB_URL, DB_USER, DB_PASSWORD	Conexão com o Oracle (os valores ficam só no painel da plataforma)
API_URL	Endereço público da API (usado para montar as URLs de imagens e evidências)
CORS_ORIGINS	Endereço do site (Vercel) autorizado a chamar a API
PORT	Porta do servidor (definida pela plataforma)